

➤ Literals.

1. Explain Integer literal caching.

Integer literal caching is a JVM optimization where **Integer** objects with values from **-128 to 127 are cached and reused** instead of creating new objects every time.

This applies to the **Integer wrapper class**, not to the primitive `int`.

When you write:

```
Integer a = 100;
```

The compiler translates it to:

```
Integer a = Integer.valueOf(100);
```

- `Integer.valueOf()` checks a **cache**
- If value is between **-128 and 127**, it returns the **same cached object**
- Otherwise, it creates a **new Integer object**

2. `final int x = 10; byte b = x;` ✓ Compiles! Why

Java allows assigning a final int compile-time constant to a smaller primitive type if the value is within the target type's range.

➤ Arrays.

1. Where are arrays stored in JVM memory?

In Java, every array (of primitives or objects) is an instance of a class. All arrays implicitly extend `java.lang.Object`.

```
int[] arr = new int[5];
```

```
System.out.println(arr instanceof Object); // true
```

Because they are objects, **arrays are allocated on the Heap**.

The **reference variable** (e.g., `arr`) is stored based on its scope:

- **Local variable** → Stack
- **Instance variable** → Heap (inside the object)
- **Static variable** → Method Area / Metaspace

Example:

```
void method() {  
    int[] arr = new int[3];  
}
```

- ```
}

```
- arr (reference) → **Stack**
  - new int[3] (actual array object) → **Heap**

## 2. Difference between primitive array and object array internally?

Internally, primitive arrays store actual values inside the array object, while object arrays store references to separate heap objects, leading to higher memory usage and slower access.

| For primitive arrays:                                                                 | For object arrays:                                                                                                                                        |
|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>int[] a = {10, 20, 30};  Heap └─ int[] object    ├── 10    ├── 20    └─ 30</pre> | <pre>String[] s = new String[3]; s[0] = "Java";  Heap └─ String[] object    ├── ref1 ──► "Java"    ├── null    └─ null    └─ String object ("Java")</pre> |

## 3. Why is array access $O(1)$ ?

Array access is  $O(1)$  because the memory address of any element can be calculated directly using the base address and index, without iteration.

$$\text{address}(\text{arr}[i]) = \text{base\_address} + (i \times \text{size\_of\_element})$$

So for: arr[3]

JVM does: base + (3 × 4 bytes)

- No loop, no search, no traversal
- Just one arithmetic operation

Why This Guarantees  $O(1)$

Time does not depend on array size. Accessing arr[0] or arr[1\_000\_000] takes the same time. Hence constant time →  $O(1)$

## 4. What is ArrayStoreException?

**ArrayStoreException** is a **runtime exception** thrown by the JVM when you try to store an **incompatible type** into an array. It occurs **only with object arrays**, not primitive arrays.

```
Number[] nums = new Integer[3]; // allowed (array covariance)
nums[0] = 3.14; // ❌ ArrayStoreException
```

5. `Arrays.copyOf()`, `System.arraycopy()` and copying one by one which is the best approach and why?

I prefer `System.arraycopy()` because it is a native method and faster.  
If readability and resizing are needed, I use `Arrays.copyOf()`.  
I use manual copying only when I need custom logic during the copy.”

## ➤ Var-args.

1. Real-World Usage of Varargs?

Logging frameworks (`log.info(String, Object...)`), `Arrays.asList(T...)`, JUnit assertions, Utility/helper methods

2. Arrays vs Var-args?

Varargs exist to make method calls simpler, cleaner, and more

## ➤ Instance of.

1. Is `instanceof` a compile-time or runtime check??

Runtime check. The decision is made based on the actual object in memory, not the reference type

2. Can we use `instanceof` with interfaces?

```
List list = new ArrayList(); System.out.println(list instanceof List)
```

3. What happens if superclass reference points to subclass object?

`instanceof` checks actual object, not reference.  
`Parent p = new Child();`  
`p instanceof Child; // true`

4. 

```
class Test {
 private Test() {}
}
Test t = Test.class.newInstance();
```

IllegalAccessException

## ➤ ClassNotFoundException vs ClassDefNotFoundError.

ClassNotFoundException occurs when the class is not found at runtime during dynamic loading, while

NoClassDefFoundError occurs when the class was present at compile time but is missing at runtime

## ➤ Oops Concept.

- Data Hiding

1. Data Hiding and Immutability  
Immutability is **strong data hiding**.

```
final class User {
 private final String name;
 public User(String name) {
 this.name = name;
 }
 public String getName() {
 return name;
 }
}
```

2. Can we hide methods in Java?

✗ Instance methods → **Cannot be hidden**

✓ Static methods → **Can be hidden**

```
class A {
 static void test() {
 }
}

class B extends A {
 static void test() {} // Method hiding
}
```

- Abstraction

1. How is Abstraction achieved in Java?

Using: Abstract classes and Interfaces

- **Encapsulation**

1. Is Encapsulation possible without getters and setters?  
=> yes it is possible

```
Class Counter{
 private int counter;
 public void increment(){
 count++;
 }
}
```

2. Encapsulation vs POJO vs JavaBean

POJO → simple class

JavaBean → encapsulated + getters/setters + no-arg constructor

- **Inheritance(Is A Relationship)**

1. What is the Diamond Problem?

Occurs when a class inherits from two classes having the same method, creating ambiguity.

👉 Java avoids this by disallowing multiple class inheritance.

2. Can constructors be inherited?

✗ No

But parent constructors are **invoked** using `super()`.

3. Inheritance vs Composition

| <b>Inheritance</b> | <b>Composition</b> |
|--------------------|--------------------|
| IS-A relationship  | HAS-A relationship |
| Tight coupling     | Loose coupling     |
| Less flexible      | More flexible      |

- **Has A Relationship**

1. HAS-A vs IS-A relationship?

**IS-A**

Inheritance

extends

Tight coupling

Less flexible

**HAS-A**

Association

Object reference

Loose coupling

More flexible

**2. Composition vs Aggregation****Feature**

Ownership

Object lifecycle

Coupling

Example

**Composition**

Strong

Dependent

High

Car–Engine

**Aggregation**

Weak

Independent

Low

Team–Player

**3.**